

Mini Chappy! をつくろう

画面づくり × AI × ちょっと不思議な「道具」呼び出し

You: 今何時?

AI: 15:32だよ!
(本物の時刻)

送信 ▶

今日の旅路：3つのステージ



MiniChappy. py を書き換えながら、3つのステージを順番に進みます。

はじめに配る、3つのファイル



MiniChappy. py

画面の設計図。まだシンプルなチャットWindow。



ai_communicate. py

AIとお話する係。今はまだ普通に会話するだけ。



ai_tools. py

AIに渡す「工具箱」。今は時計ツールが1つ入っている。

MiniChappy. py

呼び出す
→

ai_communicate. py

呼び出す
→

ai_tools. py

STAGE A - 1

画面をキレイにしよう

MiniChappy.py にコードを足して、色・フォント・レイアウトを整える。少しタイピング練習！

```
29 # ウィンドウの作成
30 root = tk.Tk()
31 root.title("Mini Chappy!")
32 root.resizable(False, False) # ウィンドウのサイズ変更を無効化 (固定)
33 root.geometry("600x500")
34 cf=("Helvetica", 14)
35
36 # チャットの履歴を表示するテキストエリア
37 chat_box = tk.Text(root, width=50, height=15, font=cf, bg="#f0f8ff")
38 chat_box.pack(padx=10, pady=10, fill=tk.BOTH, expand=True)
39
40 # 入力欄とボタンを横に並べるためのフレーム
41 frame = tk.Frame(root)
42 frame.pack(padx=10, pady=10, fill=tk.X)
43
44 # メッセージを入力する入力欄
45 entry = tk.Entry(frame, font=cf, bg="#ffffe0")
46 entry.pack(side=tk.LEFT, fill=tk.X, expand=True, padx=(0, 10))
47
48 # 送信ボタン
49 send_btn = tk.Button(frame, text="送信", command=send_message, width=12, font=cf, bg="#87CEFA")
50 send_btn.pack(side=tk.RIGHT)
51
52 # アプリを起動したままにする
```

root.geometry("600x500")
cf=("Helvetica", 14)

, font=cf, bg="#f0f8ff"

, font=cf, bg="#ffffe0"

, font=cf, bg="#87CEFA"

色コードを打つだけで画面がガラッと変わる！

STAGE A - 2

ボタンに「動き」を吹き込もう

send_message() の中に仕掛けを追加して、待ち時間の間もボタンが生きて見えるようにする。

```
7  ...if not user_text:
8  ...    return
9
10 ...send_btn.config(text="問い合わせ中...", state=tk.DISABLED, bg="#ffb347")
11
12 ...# 画面に自分の発言を表示
13 ...chat_box.insert(tk.END, "You: " + user_text + "\n")
14 ...entry.delete(0, tk.END)
15 ...chat_box.see(tk.END) # 自動で一番下までスクロール
16
17 ...# UIの変更を即座に画面に反映させる
18 ...root.update()
19
20 ...# AIからの回答待ちをシミュレート（後でAPI通信の処理が入ります）
21 ...time.sleep(1) # 1秒待機
22
23 ...# AIの返事を表示
24 ...chat_box.insert(tk.END, "AI: 準備中です...\n\n")
25 ...chat_box.see(tk.END)
26
27 ...send_btn.config(text="送信", state=tk.NORMAL, bg="#87CEFA")
```

, state=tk.DISABLED, bg="#ffb347"

ボタンの無効化

ボタンの有効化

, state=tk.NORMAL, bg="#87CEFA"

STAGE B

本物のAIとつなげよう

追記するのはたったの4行。AIとの通信・会話履歴の管理は `ai_communicate.py` が全部やってくれる。

```
1 import tkinter as tk
2 import time # 待機時間をシミュレートする
3 import ai_communicate as ai
4
5 def send_message():
```

`ai_communicate.py` を `ai` という名前で利用する宣言

```
import ai_communicate as ai
```

```
18 # UIの変更を即座に画面に反映させる
19 root.update()
```

```
answer = ai.message(user_text)
```

字下げは重要！

```
20 # AIからの回答待ちをシミュレート (後でAPI)
21 answer = ai.message(user_text)
```

`ai_communicate.py` の `message` 関数を利用
引数: `user_text` (入力した文字列)

```
24 # AIの返事を表示
```

```
25 chat_box.insert(tk.END, "AI: " + answer + chr(10) + chr(10))
26 chat_box.see(tk.END)
```

aiからの応答 (`answer`) を
chatboxに書く

```
27
28 send_btn.
```

```
chat_box.insert(tk.END, "AI: " + answer + chr(10) + chr(10))
```

```
29
30 # ウィンドウのトリック
```

```
53 ai.initialize()
```

```
ai.initialize()
```

```
54
55 # アプリを起動したままにする
56 root.mainloop()
```

「準備中です…」の固定セリフが、
本物のAIの返事になる瞬間！

Function Callingってなに？

AIは本物の時計を持っていない。だから「今何時？」と聞かれたら、時計係（ツール）に確認しに行く。



AI係に道具を持たせよう その1

ai_communicate.py を直接拡張する。道具を実際に動かす処理は ai_tools.py が全部やってくれる。

```
9 msgs = [  
10     {"role": "system",  
11     "content": "日付や時間を聞かれた場合、必ず'get_current_time'ツールを使って回答すること。"  
12     }  
13 ]
```

```
msgs = [  
    {"role": "system",  
     "content": "日付や時間を聞かれた場合、必ず'get_current_time'ツールを使って回答すること。"  
    }  
]
```

```
33 # 2. AIに履歴を渡して返答をもらう  
34 api_response = client.chat.completions.create(  
35     model = "llama3.2:3b",  
36     messages = msgs,  
37     tools = ai_tools.tools,  
38     # tool choice = "required"  
39 )
```

行頭の#を削除します
※tool_choice側は一旦残しておきます

「道具を渡す」「結果を受け取る」だけ。難しい処理は ai_tools.py にお任せ！

AI係に道具を持たせよう その2

ai_communicate.py を直接拡張する。道具を実際に動かす処理は ai_tools.py が全部やってくれる。

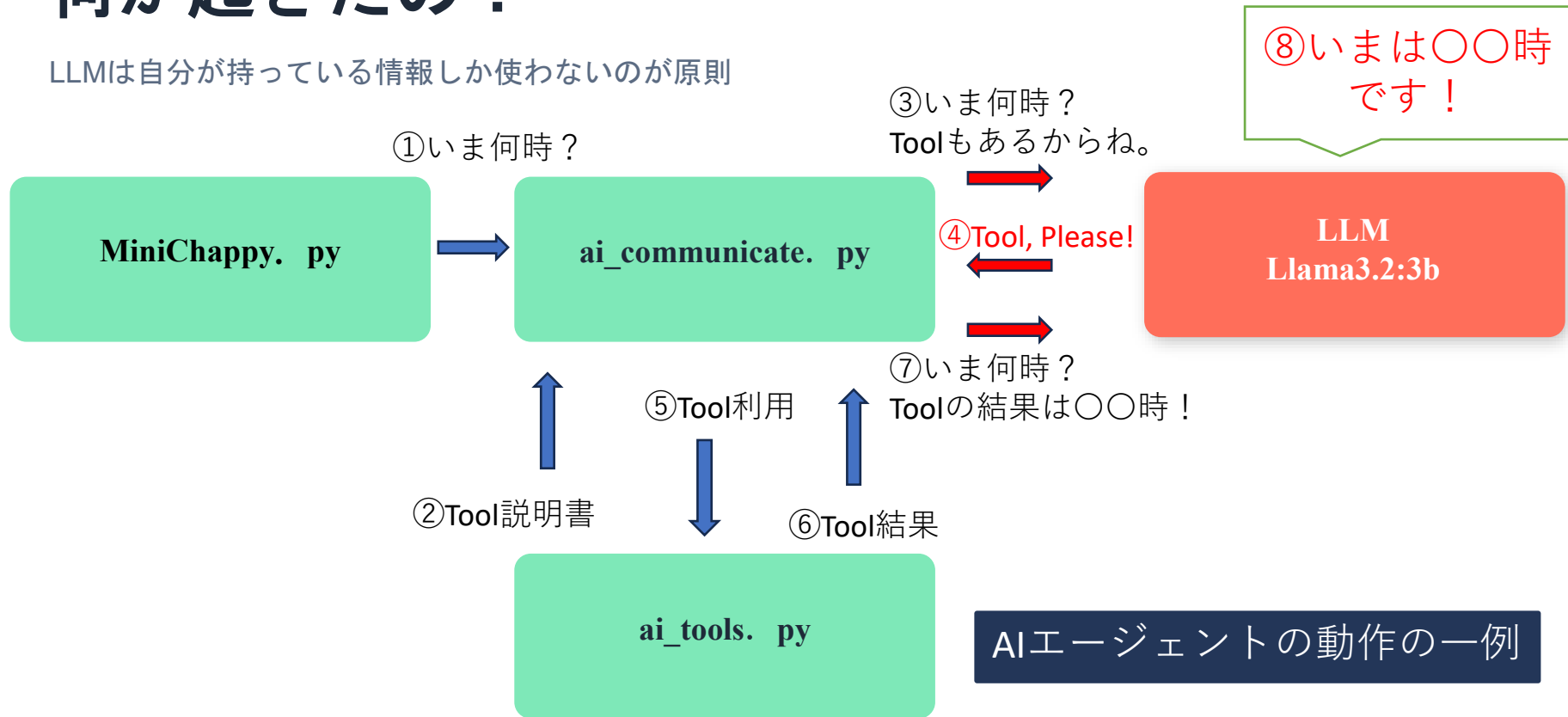
```
39 ... # 3. 応答データから「メッセージオブジェクト」を取り出す
40 ... assistant_msg = ap final_msg = ai_tools.handle_tool_call(client, "llama3.2:3b", msgs, assistant_msg)
41 ...
42 ... final_msg = ai_tools.handle_tool_call(client, "llama3.2:3b", msgs, assistant_msg)
43 ...
44 ... # 4. メッセージオブジェクトから「テキスト部分」だけを取り出して履歴に追加
45 ... reply_text = final_msg.content
46 ... msgs.append({"role": "assistant", "content": reply_text})
47 ...
48 ... # 5. 画面にAIの返答を通知
49 ... return str(reply_text)
```

reply_text = final_msg.content

「道具を渡す」「結果を受け取る」だけ。難しい処理は ai_tools.py にお任せ！

何が起きたの？

LLMは自分が持っている情報しか使わないのが原則



まとめ & おうちでチャレンジ

- ✓ STAGE A 画面をつくる
- ✓ STAGE B AIとつなぐ
- ✓ STAGE C 道具を持たせる

おうちでチャレンジ

- 色やフォントを自分好みに変える
- AIにいろいろな質問を試みる
- 新しい道具を追加する（電卓・天気など）



本日のデータは以下にあります。

<https://github.com/hide-work/Hagoromo-OC-2026>

お疲れさまでした！

自宅のPCを使って実験！

1

Windows PC & Python の準備

まずはWindows PCを用意。Pythonも[公式サイト](#)からインストールしておこう。

2

Ollama で脳 (モデル) をダウンロード

[ollama.com](#) から本体をインストール。コマンドプロンプトで `ollama pull llama3.2:3b` を実行！

3

OpenAIライブラリのインストール (Pythonのライブラリ)

AIを操作する「工具箱」を準備。コマンドプロンプトで `pip install openai` と入力しよう。

4

ai_communicate.pyを修正

下図のとおり、URLをlocalhostに修正

```
20  ... # AIに接続するための準備
21  ... client = OpenAI(
22  ...     base_url = "http://localhost:11434/v1",
23  ...     api_key = "dummy"
24  ... )
```